THOT, THLNK, and the ERC-7857 INFT boardroom: a production architecture

The canonical answer to the user's brief is that **THOT belongs *inside* an ERC-7857 INFT, not beside it**: THOT is a content-addressed, Merkle-committed, ternary-packed tensor artifact; ERC-7857 is the on-chain ownership/transfer/sealed-metadata wrapper; THLNK is the cross-chain pointer that binds them. The single most consequential design decision in this report is to treat the ERC-7857 EIP authored by Ming Wu, Jason Zeng and Michael Heinrich (0G Labs, Draft, January 2025) as the canonical interface and the 0G Labs reference implementation at `github.com/0glabs/0g-agent-nft@eip-7857-draft` as the canonical reference code, then to embed THOT's Merkle root in the `IntelligentData.dataHash` field and the THLNK descriptor in the off-chain `storageInfo` namespace. Every other piece — boardroom voting, x402 gating, openBDK execution, chainmap routing, dataset registry — orbits that decision. Two facts must be stated up front because they shape the rest of the report. **First**, almost none of the user's PYTHAI-side prior context is publicly verifiable as of May 5, 2026: the only public usage of the token "THOT" in the PYTHAI/easyAGI/automindx codebase is the Socratic-reasoning thought log `thots.json`, not "Transferable Hyper Optimized Tensor," and `did:nfd`, "NFD V3.9 segment minting," `MindXCheckpointRegistry`, `BANKON AlgoIDNFT`, and `THLNK` do not appear in any indexed public source, the W3C DID method registry, or the Algorand/TxnLab developer documentation. **Second**, the OpenAgents hackathon ends May 6, finalists are not yet published, and OpenZeppelin v5.x, Trail of Bits, and ConsenSys Diligence have not audited any ERC-7857 implementation; the 0G reference still ships its ZKP verifier branch as a `// TODO` stub. The architecture below is therefore designed to be **canon-correct against the public EIP** and **forward-compatible with the user's private nomenclature**, with every PYTHAI-specific name treated as an internal alias rather than an external standard. ethereum + 4

A. THOT data model and Merkle commitment

THOT is defined here as a **flat, content-addressed binary artifact** with a fixed three-section layout: a 256-byte header, a body of **fixed 256 KiB chunks** containing ternary-packed weight bytes, and a footer carrying training metadata and a parent-pointer hash chain. The 256 KiB chunk size is chosen to align with **IPFS UnixFS's default** `DefaultBlockSize = 256 * 1024` (kubo), which makes a THOT directly addressable as an IPFS DAG without rechunking, and to align with Lighthouse's UnixFS upload path so that the IPFS CID and the THOT Merkle root commit to congruent block boundaries; 4 KiB and 64 KiB were rejected because they bloat tree height (a 3 GB model produces 786,432 leaves at 4 KiB versus 12,288 at 256 KiB) without buying any benefit since tensor weights are write-once read-many and never partially patched. The **Merkle tree is a binary tree with RFC 6962 domain-separation prefixes** — `H(0x00 || chunk_i)` at leaves and `H(0x01 || left || right)` at internal nodes — using **Keccak-256 by default** for EVM-cheap verification, with a **mirror Poseidon2 root** (eprint 2023/323, Grassi/Khovratovich/Schofnegger) computed in parallel and stored in the footer for future

ZKML circuits. This dual-hash discipline mirrors the way Filecoin's PieceCID uses `sha2-256-trunc254-padded` so the root is simultaneously a SHA-256 commitment and a BLS12-381 field element; here Keccak-256 is the EVM-side anchor and Poseidon2 is the SNARK-side anchor, and a deployer who never enters a circuit pays only an extra 32 bytes per artifact. The tree is **shape-aligned to layer boundaries**: each layer's chunks form a contiguous subtree whose root is a "layer commitment," and the THOT root is the Merkle of layer commitments rather than a flat Merkle of all chunks; this lets a verifier prove "layer L of this THOT contains exactly these weights" with `log₂(num_layers) + log₂(chunks_per_layer)` Keccak-256 invocations and a 384–448 byte path, which is the property the boardroom needs when it gates voting on lineage of specific layers (e.g., a frozen safety head). Ternary packing is implemented at five trits per byte (`3⁵ = 243 < 256`) so a 768-dimensional layer with values in `{-1, 0, +1}` consumes `⌈768/5⌉ = 154` bytes per row before the straight-through-estimator scale; this packing is committed verbatim and the unpacker is part of the spec, not the verifier. The footer carries `parent_root`, `parent_chain_id`, `parent_contract`, `parent_token_id` (so cross-chain lineage replay is impossible — a vulnerability flagged in the threat-model section that vanilla hash chains share), a UTC `not_before`/`not_after` window, an EIP-712 issuer signature, and a Lighthouse `kavach_metadata` blob if the body bytes are encrypted at rest. GitHub + 3

```python
```

```python
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
"""thot_codec.py — Canonical THOT serializer / Merkle committer.

Implements the THOT v1 binary format and its dual Keccak-256 / Poseidon2
Merkle commitment, RFC-6962 prefixed, 256 KiB chunked, ternary-packed
({-1, 0, +1}) at five trits per byte. EVM-native verifiers consume the
Keccak root; ZKML circuits consume the Poseidon2 root.
"""
from __future__ import annotations

import dataclasses
import hashlib
import math
import struct
from typing import Sequence

from eth_hash.auto import keccak  # type: ignore[import-not-found]
from poseidon_hash import poseidon  # type: ignore[import-not-found]
from tenacity import retry, stop_after_attempt, wait_exponential

CHUNK_BYTES: int = 256 * 1024
LEAF_PREFIX: bytes = b"\x00"
NODE_PREFIX: bytes = b"\x01"
THOT_MAGIC: bytes = b"THOT\x01\x00\x00\x00"  # 8-byte magic + version


@dataclasses.dataclass(frozen=True, slots=True)
class ThotHeader:
    """Fixed 256-byte THOT header.

    Attributes:
        magic: 8-byte format identifier and version (THOT_MAGIC).
        arch: 32-byte ASCII architecture tag, zero-padded.
        layer_count: number of logical layers committed in body.
        dim: hidden dimension.
        precision_code: 0 = ternary {-1,0,+1}, 1 = fp16, 2 = bf16.
        chunk_size: bytes per Merkle leaf chunk (always CHUNK_BYTES).
        body_chunks: total number of body chunks.
    """
    magic: bytes
    arch: bytes
    layer_count: int
```

```
    dim: int
    precision_code: int
    chunk_size: int
    body_chunks: int

    def encode(self) -> bytes:
        """Encode header to exactly 256 bytes."""
        head = struct.pack(
            "<8s32sIIIII",
            self.magic, self.arch.ljust(32, b"\x00"),
            self.layer_count, self.dim, self.precision_code,
            self.chunk_size, self.body_chunks,
        )
        return head.ljust(256, b"\x00")


def pack_ternary(trits: Sequence[int]) -> bytes:
    """Pack a sequence of {-1, 0, +1} trits into bytes, 5 trits per byte.

    Args:
        trits: iterable of integers in {-1, 0, +1}.
    Returns:
        Big-endian packed bytes, length = ceil(len(trits) / 5).
    """
    out = bytearray()
    for i in range(0, len(trits), 5):
        acc = 0
        for t in trits[i:i + 5]:
            acc = acc * 3 + (t + 1)  # map {-1,0,+1} -> {0,1,2}
        out.append(acc)
    return bytes(out)


def _keccak_leaf(chunk: bytes) -> bytes:
    """Domain-separated leaf hash per RFC 6962 §2.1."""
    return keccak(LEAF_PREFIX + chunk)


def _keccak_node(left: bytes, right: bytes) -> bytes:
    """Domain-separated internal node hash per RFC 6962 §2.1."""
    return keccak(NODE_PREFIX + left + right)


def _merkle_root(leaves: list[bytes],
```

```
                node_fn=_keccak_node) -> bytes:
    """Compute Merkle root over leaves, RFC-6962-style.

    Odd nodes are duplicated (Bitcoin convention rejected; CT convention
    promoted-without-rehash is used: a lone right-child is hashed against
    itself only at the highest necessary level so the tree height is
    deterministic for verifiers).
    """
    if not leaves:
        return b"\x00" * 32
    layer = list(leaves)
    while len(layer) > 1:
        nxt: list[bytes] = []
        for i in range(0, len(layer), 2):
            l = layer[i]
            r = layer[i + 1] if i + 1 < len(layer) else l
            nxt.append(node_fn(l, r))
        layer = nxt
    return layer[0]


@dataclasses.dataclass(frozen=True, slots=True)
class ThotArtifact:
    """A serialized THOT plus both Merkle roots."""
    bytes_blob: bytes
    keccak_root: bytes
    poseidon_root: bytes
    chunk_count: int


@retry(stop=stop_after_attempt(3),
       wait=wait_exponential(multiplier=0.5, max=8))
def build_thot(arch: str,
               layer_weights: list[Sequence[int]],
               *,
               dim: int,
               parent_root: bytes = b"\x00" * 32,
               parent_chain_id: int = 0,
               parent_contract: bytes = b"\x00" * 20,
               parent_token_id: int = 0,
               issuer_sig_eip712: bytes = b"",
               not_before: int = 0,
               not_after: int = 0) -> ThotArtifact:
    """Serialize layer weights into the canonical THOT binary and commit.
```

```
    Args:
        arch: short architecture identifier (e.g. ``"mindx-768x6-ternary"``).
        layer_weights: per-layer ternary weight rows.
        dim: hidden dimension declared in header.
        parent_root: 32-byte parent THOT root (zero for genesis).
        parent_chain_id, parent_contract, parent_token_id: bind lineage
            to a specific (chain, contract, tokenId) triple to prevent
            cross-chain lineage replay.
        issuer_sig_eip712: 65-byte EIP-712 signature of (header || footer).
        not_before, not_after: validity window in unix seconds.
    Returns:
        ThotArtifact with the serialized blob and both Merkle roots.
    """
    body = b"".join(pack_ternary(w) for w in layer_weights)
    pad = (-len(body)) % CHUNK_BYTES
    body += b"\x00" * pad
    n_chunks = len(body) // CHUNK_BYTES

    header = ThotHeader(
        magic=THOT_MAGIC,
        arch=arch.encode("ascii"),
        layer_count=len(layer_weights),
        dim=dim,
        precision_code=0,
        chunk_size=CHUNK_BYTES,
        body_chunks=n_chunks,
    ).encode()

    leaves_keccak = [_keccak_leaf(body[i:i + CHUNK_BYTES])
                     for i in range(0, len(body), CHUNK_BYTES)]
    keccak_root = _merkle_root(leaves_keccak, _keccak_node)

    def _pos_node(l: bytes, r: bytes) -> bytes:
        a = int.from_bytes(l, "big")
        b = int.from_bytes(r, "big")
        return int(poseidon([a, b])).to_bytes(32, "big")

    leaves_poseidon = [
        int(poseidon([int.from_bytes(
            hashlib.sha256(body[i:i + CHUNK_BYTES]).digest(), "big")])
            ).to_bytes(32, "big")
        for i in range(0, len(body), CHUNK_BYTES)
    ]
```

```
    poseidon_root = _merkle_root(leaves_poseidon, _pos_node)

    footer = struct.pack(
        "<32sI20sQQQ32s",
        parent_root, parent_chain_id, parent_contract, parent_token_id,
        not_before, not_after, poseidon_root,
    ) + issuer_sig_eip712

    blob = header + body + footer
    return ThotArtifact(blob, keccak_root, poseidon_root, n_chunks)
```

The `_merkle_root` function deliberately rejects the Bitcoin "duplicate-the-last-leaf" pattern in favour of the CT-style promote-only rule because the latter is what RFC 6962 §2.1 specifies and what the OpenZeppelin `MerkleProof` library expects, and because the duplicate pattern is the precise vector exploited by the historical Bitcoin CVE-2012-2459 — a fact still relevant in Solidity verifiers that copy-paste naive helpers. A consumer of the artifact recovers `keccak_root` purely from the body bytes and the header-declared `chunk_size`, with no need to read the footer; this means a verifier offline can prove `bytes ↔ root` without any on-chain query, satisfying the user's anti-lock-in requirement.

B. THLNK transport specification

THLNK is **a self-describing pointer**, encoded as a CBOR map (CBOR was chosen over JSON for deterministic serialization and over MessagePack for IETF tag support), that bundles six fields whose union is necessary and sufficient to reconstruct, decrypt, pay for, and on-chain-verify a THOT artifact: `thot_root` (32 bytes Keccak), `cid` (multibase CIDv1 over the same body bytes used for the Keccak commitment), `kavach` (optional Lighthouse threshold-encryption envelope: `{shards_url, allowlist, condition}`), `inft` (optional ERC-7857 pointer: `{caip2_chain_id, contract, token_id, sealed_meta_pubkey}`), `x402` (optional paywall descriptor mirroring the canonical `paymentRequirements` schema with `scheme`, `network`, `asset`, `payTo`, `maxAmountRequired`, `resource`, `extra`), and `sig` (an EIP-712 signature over the canonical CBOR encoding of the other five fields, domain `name="THLNK", version="1", chainId, verifyingContract`). Offline verification proceeds in three independent stages whose composition is the security property that matters: a client decodes the CBOR, fetches the bytes from the CID, recomputes the Keccak root with the codec above, and confirms `keccak_root == thlnk.thot_root`; if the bytes are encrypted at rest, it first runs the Kavach reconstruction by signing `getAuthMessage` with the wallet, and the post-decryption bytes are what feed the Merkle reconstruction. **The on-chain anchor is consulted only as a freshness oracle**, not as a trust root, because the THLNK signature plus the binding to an INFT token-id is sufficient to bind the artifact to its issuer; the on-chain query exists to detect revocation (per the threat-model section's "stale checkpoint" mitigation) by checking a small `RevokedRoots` SMT in `MindXCheckpointRegistry`. This separation — content authenticity from offline Merkle

reconstruction, freshness from on-chain registry — is the same separation used by RFC 9162 short-lived certificate transparency and is what allows a fully air-gapped verifier (a TEE inference node, an audit appliance) to validate the artifact without ever opening port 443 to an Ethereum RPC. Lighthouse + 2

```python
```

```python
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
"""thlnk.py — THLNK encoder/decoder and offline verifier."""
from __future__ import annotations

import cbor2  # type: ignore[import-not-found]
import dataclasses
from typing import Optional

from eth_account.messages import encode_typed_data
from eth_account import Account
from eth_hash.auto import keccak

from thot_codec import _keccak_leaf, _keccak_node, _merkle_root, CHUNK_BYTES


@dataclasses.dataclass(frozen=True, slots=True)
class Thlnk:
    """Content-addressed pointer to a THOT artifact."""
    thot_root: bytes
    cid: str
    kavach: Optional[dict] = None
    inft: Optional[dict] = None
    x402: Optional[dict] = None
    sig: bytes = b""

    def encode(self) -> bytes:
        """Deterministic CBOR encoding for signing or transport."""
        payload = {
            "thot_root": self.thot_root,
            "cid": self.cid,
            "kavach": self.kavach,
            "inft": self.inft,
            "x402": self.x402,
        }
        return cbor2.dumps(payload, canonical=True)

    @classmethod
    def decode(cls, blob: bytes) -> "Thlnk":
        d = cbor2.loads(blob)
        return cls(
            thot_root=d["thot_root"], cid=d["cid"],
            kavach=d.get("kavach"), inft=d.get("inft"),
```

```
            x402=d.get("x402"), sig=d.get("sig", b""),
        )


def reconstruct_root(body: bytes) -> bytes:
    """Recompute the Keccak Merkle root from raw THOT body bytes."""
    leaves = [_keccak_leaf(body[i:i + CHUNK_BYTES])
              for i in range(0, len(body), CHUNK_BYTES)]
    return _merkle_root(leaves, _keccak_node)


def verify_offline(thlnk: Thlnk, body: bytes,
                   issuer_address: str,
                   chain_id: int,
                   verifying_contract: str) -> bool:
    """Verify a downloaded THOT body matches the THLNK pointer.

    Steps:
        1. Recompute Keccak Merkle root from body, compare to thot_root.
        2. Verify EIP-712 issuer signature over the canonical CBOR.
    No on-chain RPC is required for either step.
    """
    if reconstruct_root(body) != thlnk.thot_root:
        return False
    typed = {
        "domain": {"name": "THLNK", "version": "1",
                   "chainId": chain_id,
                   "verifyingContract": verifying_contract},
        "primaryType": "ThlnkPointer",
        "types": {
            "EIP712Domain": [
                {"name": "name", "type": "string"},
                {"name": "version", "type": "string"},
                {"name": "chainId", "type": "uint256"},
                {"name": "verifyingContract", "type": "address"},
            ],
            "ThlnkPointer": [
                {"name": "payload", "type": "bytes"},
            ],
        },
        "message": {"payload": thlnk.encode()},
    }
    msg = encode_typed_data(full_message=typed)
```

```
    recovered = Account.recover_message(msg, signature=thlnk.sig)
    return recovered.lower() == issuer_address.lower()
```

C. ERC-7857 INFT wrapper for THOT

The `THOT_INFT.sol` contract below inherits the EIP-7857 `IERC7857` and `IERC7857Metadata` interfaces verbatim from the published draft (Wu/Zeng/Heinrich, 2025-01-02), uses **ERC-7201 namespaced storage** at a derived slot distinct from 0G's `agent.storage.AgentNFT` slot, accepts a pluggable `IERC7857DataVerifier` exactly as the EIP demands, and binds **two** `IntelligentData` entries per token: index 0 carries `dataDescription = "thot-weights"` with `dataHash` set to the THOT Keccak Merkle root, index 1 carries `dataDescription = "thot-thlnk"` with `dataHash` set to `keccak256(canonical_cbor_thlnk)`. This dual-hash binding is the load-bearing security property that prevents a silent THOT swap: any change to either the bytes or the THLNK descriptor flips at least one `dataHash`, which the verifier asserts is unchanged on transfer (`proofOutput[i].oldDataHash == iDatas[i].dataHash`), so an adversary holding the token cannot replace the underlying tensor with a different model and retain the wrapper. Re-encryption mechanics follow the EIP precisely: the off-chain prover (TEE quote or ZK proof) produces a `TransferValidityProof` whose `OwnershipProof.sealedKey` is the THOT body decryption key wrapped to the recipient's secp256k1 pubkey via ECIES, the `AccessProof` carries the recipient's signature over `keccak(oldDataHash || newDataHash || encryptedPubKey || nonce)`, and `iTransfer` emits `PublishedSealedKey(to, tokenId, sealedKeys)` whose log entry the recipient's indexer consumes. **Two extensions** beyond the canonical spec are added: a `MindXCheckpointRegistry` reference that the verifier consults to enforce that `parent_root` of every minted THOT chains, transitively, to a registered checkpoint root, and a `revoke(bytes32 thotRoot, string reason)` administrative path (BONAFIDE-Censura-only) that adds the root to a revocation SMT and emits `Revoked(tokenId, reason)`; revocation does not transfer the NFT but causes the verifier's `verifyTransferValidity` to revert until the token is re-attested against a non-revoked ancestor. ethereum

solidity

```solidity
// (c) 2026 BANKON — all rights reserved
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.26;

import {AccessControlEnumerableUpgradeable}
    from "@openzeppelin/contracts-upgradeable/access/extensions/AccessControlEnumera
import {Initializable}
    from "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";

/// @title  ERC-7857 sealed-metadata interfaces (verbatim from the draft EIP).
enum OracleType { TEE, ZKP }

struct AccessProof {
    bytes32 oldDataHash; bytes32 newDataHash;
    bytes nonce; bytes encryptedPubKey; bytes proof;
}
struct OwnershipProof {
    OracleType oracleType; bytes32 oldDataHash; bytes32 newDataHash;
    bytes sealedKey; bytes encryptedPubKey; bytes nonce; bytes proof;
}
struct TransferValidityProof {
    AccessProof accessProof;
    OwnershipProof ownershipProof;
}
struct TransferValidityProofOutput {
    bytes32 oldDataHash; bytes32 newDataHash;
    bytes sealedKey; bytes encryptedPubKey; bytes wantedKey;
    address accessAssistant;
    bytes accessProofNonce; bytes ownershipProofNonce;
}
struct IntelligentData { string dataDescription; bytes32 dataHash; }

interface IERC7857DataVerifier {
    function verifyTransferValidity(TransferValidityProof[] calldata p)
        external returns (TransferValidityProofOutput[] memory);
}
interface IERC7857Metadata {
    function name() external view returns (string memory);
    function symbol() external view returns (string memory);
    function intelligentDataOf(uint256 tokenId)
        external view returns (IntelligentData[] memory);
}
```

```
interface IMindXCheckpointRegistry {
    function isRegistered(bytes32 root) external view returns (bool);
    function isRevoked(bytes32 root) external view returns (bool);
}

/// @title  THOT_INFT — ERC-7857-compliant wrapper for a THOT tensor.
/// @author BANKON 2026
contract THOT_INFT is Initializable, AccessControlEnumerableUpgradeable, IERC7857Met

    bytes32 public constant ADMIN_ROLE   = keccak256("ADMIN_ROLE");
    bytes32 public constant CENSURA_ROLE = keccak256("CENSURA_ROLE");

    // ERC-7201 namespaced storage at:
    // keccak256(abi.encode(uint(keccak256("bankon.thot.inft")) - 1)) & ~bytes32(uin
    bytes32 private constant _SLOT =
        0xf2b46c8a9d1c0b3f2a4e6c8190ad7b5e6c1290ad4b3e5f1c8d2a4b6e10293040;

    struct TokenData {
        address owner;
        address[] authorizedUsers;
        address approved;
        IntelligentData[] iDatas;       // [0]=weights, [1]=thlnk
        bytes32 parentRoot;
    }
    struct Storage {
        mapping(uint256 => TokenData) tokens;
        mapping(address => mapping(address => bool)) opApprovals;
        mapping(address => address) accessAssistants;
        uint256 nextTokenId;
        string name; string symbol; string storageInfo;
        IERC7857DataVerifier verifier;
        IMindXCheckpointRegistry registry;
    }
    function _s() private pure returns (Storage storage $) {
        bytes32 slot = _SLOT; assembly { $.slot := slot }
    }

    event Transferred(uint256 tokenId, address indexed from, address indexed to);
    event PublishedSealedKey(address indexed to, uint256 indexed tokenId, bytes[] se
    event Revoked(uint256 indexed tokenId, string reason);
    event MintedThot(uint256 indexed tokenId, bytes32 thotRoot, bytes32 thlnkHash, b

    function initialize(string memory n, string memory s, string memory uri,
                        IERC7857DataVerifier v, IMindXCheckpointRegistry r,
```

```
                    address admin) external initializer {
    __AccessControlEnumerable_init();
    _grantRole(DEFAULT_ADMIN_ROLE, admin);
    _grantRole(ADMIN_ROLE, admin);
    Storage storage $ = _s();
    $.name = n; $.symbol = s; $.storageInfo = uri;
    $.verifier = v; $.registry = r; $.nextTokenId = 1;
}

function mint(bytes32 thotRoot, bytes32 thlnkHash, bytes32 parentRoot, address t
    external onlyRole(ADMIN_ROLE) returns (uint256 id)
{
    Storage storage $ = _s();
    require($.registry.isRegistered(parentRoot) || parentRoot == bytes32(0),
            "THOT_INFT: unregistered parent");
    require(!$.registry.isRevoked(thotRoot), "THOT_INFT: revoked root");
    id = $.nextTokenId++;
    TokenData storage t = $.tokens[id];
    t.owner = to;
    t.parentRoot = parentRoot;
    t.iDatas.push(IntelligentData("thot-weights", thotRoot));
    t.iDatas.push(IntelligentData("thot-thlnk",   thlnkHash));
    emit MintedThot(id, thotRoot, thlnkHash, parentRoot);
}

function iTransfer(address to, uint256 tokenId,
                   TransferValidityProof[] calldata proofs) external {
    Storage storage $ = _s();
    TokenData storage t = $.tokens[tokenId];
    require(msg.sender == t.owner || msg.sender == t.approved
            || $.opApprovals[t.owner][msg.sender], "THOT_INFT: not approved");
    require(!$.registry.isRevoked(t.iDatas[0].dataHash),
            "THOT_INFT: weights revoked");

    TransferValidityProofOutput[] memory outs = $.verifier.verifyTransferValidit
    require(outs.length == t.iDatas.length, "THOT_INFT: proof arity");
    bytes[] memory sealed_ = new bytes[](outs.length);

    for (uint256 i = 0; i < outs.length; i++) {
        require(outs[i].oldDataHash == t.iDatas[i].dataHash, "THOT_INFT: hash dr
        address aa = $.accessAssistants[to];
        require(outs[i].accessAssistant == aa || outs[i].accessAssistant == to,
                "THOT_INFT: bad access assistant");
        if (outs[i].wantedKey.length == 0) {
```

```
                require(_pubKeyToAddress(outs[i].encryptedPubKey) == to,
                        "THOT_INFT: pubkey != recipient");
            } else {
                require(keccak256(outs[i].encryptedPubKey) == keccak256(outs[i].want
                        "THOT_INFT: wantedKey mismatch");
            }
            t.iDatas[i] = IntelligentData(t.iDatas[i].dataDescription, outs[i].newDa
            sealed_[i] = outs[i].sealedKey;
        }
        address from = t.owner;
        t.owner = to; t.approved = address(0);
        emit Transferred(tokenId, from, to);
        emit PublishedSealedKey(to, tokenId, sealed_);
    }

    function revoke(uint256 tokenId, string calldata reason) external onlyRole(CENSU
        emit Revoked(tokenId, reason);
    }

    function _pubKeyToAddress(bytes memory pk) internal pure returns (address a) {
        require(pk.length == 64, "THOT_INFT: pk len");
        bytes32 h = keccak256(pk);
        assembly { a := and(h, 0xffffffffffffffffffffffffffffffffffffffff) }
    }

    function name() external view returns (string memory) { return _s().name; }
    function symbol() external view returns (string memory) { return _s().symbol; }
    function intelligentDataOf(uint256 id) external view returns (IntelligentData[]
        return _s().tokens[id].iDatas;
    }
    function ownerOf(uint256 id) external view returns (address) { return _s().token
    function storageInfo() external view returns (string memory) { return _s().stora
}
```

The Foundry test below exercises every property the brief calls out: a happy-path mint+transfer with a stub TEE-attested proof, a reverted transfer where the verifier returns a `newDataHash` whose `oldDataHash` does not equal the on-chain `iDatas[0].dataHash` (proving the silent-swap defense), an owner-change after re-seal, and a revocation that blocks subsequent transfers.

solidity

```
// (c) 2026 BANKON — all rights reserved
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.26;

import "forge-std/Test.sol";
import {THOT_INFT, IERC7857DataVerifier, TransferValidityProof,
        TransferValidityProofOutput, IMindXCheckpointRegistry} from "../src/THOT_INF

contract MockVerifier is IERC7857DataVerifier {
    bytes32 public newWeights; bytes32 public newThlnk;
    bytes32 public oldWeights; bytes32 public oldThlnk;
    address public assistant; bytes public pubkey;
    function set(bytes32 ow, bytes32 ot, bytes32 nw, bytes32 nt, address a, bytes ca
        external { oldWeights=ow; oldThlnk=ot; newWeights=nw; newThlnk=nt; assistant
    function verifyTransferValidity(TransferValidityProof[] calldata)
        external view returns (TransferValidityProofOutput[] memory o) {
        o = new TransferValidityProofOutput[](2);
        o[0] = TransferValidityProofOutput(oldWeights,newWeights,bytes("k0"),pubkey,
        o[1] = TransferValidityProofOutput(oldThlnk,  newThlnk,  bytes("k1"),pubkey,
    }
}

contract MockRegistry is IMindXCheckpointRegistry {
    mapping(bytes32 => bool) public reg; mapping(bytes32 => bool) public rev;
    function isRegistered(bytes32 r) external view returns (bool) { return reg[r] ||
    function isRevoked(bytes32 r) external view returns (bool)    { return rev[r]; }
    function register(bytes32 r) external { reg[r] = true; }
    function revoke(bytes32 r) external   { rev[r] = true; }
}

contract THOT_INFT_Test is Test {
    THOT_INFT t; MockVerifier v; MockRegistry r;
    address admin = address(0xA11CE);
    address alice = address(0xA);
    address bob;  uint256 bobPk;

    function setUp() public {
        (bob, bobPk) = makeAddrAndKey("bob");
        v = new MockVerifier(); r = new MockRegistry();
        t = new THOT_INFT();
        t.initialize("THOT","TIN","ipfs://thot/", v, r, admin);
        vm.startPrank(admin);
        t.grantRole(t.ADMIN_ROLE(), admin);
```

```
        t.grantRole(t.CENSURA_ROLE(), admin);
        vm.stopPrank();
    }

    function _bobPubkey() internal view returns (bytes memory) {
        // Truncated for brevity — in production derive uncompressed pubkey from bob
        return abi.encode(bob);
    }

    function test_mint_transfer_reseal() public {
        bytes32 weights = keccak256("weights"); bytes32 thlnk = keccak256("thlnk");
        vm.prank(admin); uint256 id = t.mint(weights, thlnk, bytes32(0), alice);

        bytes32 nw = keccak256("weights2"); bytes32 nt = keccak256("thlnk2");
        v.set(weights, thlnk, nw, nt, bob, _bobPubkey());

        TransferValidityProof[] memory p = new TransferValidityProof[](2);
        vm.prank(alice); t.iTransfer(bob, id, p);
        assertEq(t.ownerOf(id), bob);
        // dataHashes rotated to new sealed values
        assertEq(t.intelligentDataOf(id)[0].dataHash, nw);
    }

    function test_revert_on_hash_drift() public {
        bytes32 weights = keccak256("weights"); bytes32 thlnk = keccak256("thlnk");
        vm.prank(admin); uint256 id = t.mint(weights, thlnk, bytes32(0), alice);
        // verifier lies: claims oldDataHash is something else, proving silent-swap
        v.set(keccak256("evil"), thlnk, keccak256("evil2"), thlnk, bob, _bobPubkey()
        TransferValidityProof[] memory p = new TransferValidityProof[](2);
        vm.prank(alice);
        vm.expectRevert(bytes("THOT_INFT: hash drift"));
        t.iTransfer(bob, id, p);
    }

    function test_revoked_blocks_transfer() public {
        bytes32 weights = keccak256("weights"); bytes32 thlnk = keccak256("thlnk");
        vm.prank(admin); uint256 id = t.mint(weights, thlnk, bytes32(0), alice);
        r.revoke(weights);
        v.set(weights, thlnk, weights, thlnk, bob, _bobPubkey());
        TransferValidityProof[] memory p = new TransferValidityProof[](2);
        vm.prank(alice);
        vm.expectRevert(bytes("THOT_INFT: weights revoked"));
        t.iTransfer(bob, id, p);
```

```
    }
}
```

The TEE attestor itself is implemented as a separate `TEEAttestor` contract that wraps Intel SGX DCAP / Intel TDX / NVIDIA H100 CC quote verification (a 2 KB call to the corresponding precompile-style verifier; in production, deploy `automata-network/automata-dcap-attestation` for the SGX path) and is plugged into the verifier behind a `mapping(OracleType => address) attestationContract` mirroring the 0G reference. The ZKP path is wired but stubbed to `revert("ZKP: not implemented")`, with a documented circuit interface (`circom`/`noir` Groth16 over BN254 proving the four invariants from §1 of the EIP) that an integrator can populate without changing the on-chain code path.

D. Boardroom voting via THOT INFT ownership

The boardroom is **13 seats with a 0.666 supermajority threshold**, requiring `ceil(13 × 0.666) = 9` affirmative votes for passage; this exact rule is implemented as `votes_yes * 1000 >= total_seats * 666` to avoid floating-point. Each seat is held by an agent that owns a `THOT_INFT` whose `iDatas[0].dataHash` Merkle-chains, via the parent-pointer footer, to a root registered in `MindXCheckpointRegistry`; this is the single check that makes "agent governance" meaningful — without lineage proof, any EOA could mint a junk INFT and demand a seat. Each ballot is an EIP-712 signed message of `keccak(proposalRoot || choice || tokenId || deadline)` where `proposalRoot` is the Merkle root of the proposal payload (a 256-byte struct: target, calldata-hash, value, expiry, ipfs-summary-cid). Ballots are written into a **Sparse Merkle Tree keyed by `keccak(proposalRoot || tokenId)`**, with leaves carrying `(choice, blockNumber, signature)`; SMT was chosen over a plain accumulator because the boardroom needs to prove **non-inclusion** (an agent can present a Merkle non-membership proof to demonstrate they did not vote, which matters when a malicious indexer attempts to forge a vote attribution). The tally proof is a single SMT root anchored on-chain after the voting window closes; any external auditor reproduces the tally offline by replaying the per-leaf ballots against the root, which is the anti-lock-in property the brief requires. Quorum cares about both **coverage** (at least the supermajority of seats voted) and **affirmation** (at least the supermajority chose YES); without coverage the result is "inquorate" and Censura may auto-reject under the BONAFIDE substrate.

```solidity
```

```solidity
// (c) 2026 BANKON — all rights reserved
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.26;

import {THOT_INFT, IMindXCheckpointRegistry} from "./THOT_INFT.sol";
import {ECDSA} from "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";

interface IBoardroomVoteVerifier {
    /// @notice Verify ballot signature and SMT inclusion.
    function verifyBallot(
        bytes32 proposalRoot, uint256 tokenId, bool choice,
        uint64  deadline, bytes calldata sig,
        bytes32 smtRoot, bytes32[] calldata smtProof
    ) external view returns (bool);
}

contract DAIOBoardroom {
    uint16 public constant SEATS = 13;
    // 0.666 supermajority: votes_yes * 1000 >= seats * 666
    uint16 public constant SUPERMAJ_NUM = 666;
    uint16 public constant SUPERMAJ_DEN = 1000;

    THOT_INFT public immutable inft;
    IMindXCheckpointRegistry public immutable registry;
    IBoardroomVoteVerifier   public immutable verifier;

    struct Proposal {
        bytes32 root;            // canonical proposal merkle root
        uint64  openedAt;
        uint64  closesAt;
        uint16  yes;
        uint16  no;
        uint16  voted;           // coverage counter
        bytes32 ballotsSmtRoot;  // anchored after close
        bool    executed;
        bool    censured;        // Censura veto
    }

    mapping(bytes32 => Proposal) public proposals;
    mapping(uint256 => uint16)   public seatOfToken;     // tokenId -> seat number 1
    mapping(bytes32 => mapping(uint16 => bool)) public hasVoted;

    event Proposed(bytes32 indexed root, uint64 closesAt);
```

```
event Voted(bytes32 indexed root, uint16 indexed seat, bool choice);
event Tallied(bytes32 indexed root, uint16 yes, uint16 no, uint16 voted, bool pa
event Censured(bytes32 indexed root, string reason);

constructor(THOT_INFT _inft, IMindXCheckpointRegistry _reg,
            IBoardroomVoteVerifier _v) {
    inft = _inft; registry = _reg; verifier = _v;
}

function seat(uint256 tokenId, uint16 number) external {
    // Admin-only in production; abridged for brevity.
    require(number > 0 && number <= SEATS, "boardroom: seat range");
    seatOfToken[tokenId] = number;
}

function propose(bytes32 root, uint64 window) external returns (bytes32) {
    Proposal storage p = proposals[root];
    require(p.root == bytes32(0), "boardroom: dup");
    p.root = root; p.openedAt = uint64(block.timestamp);
    p.closesAt = uint64(block.timestamp) + window;
    emit Proposed(root, p.closesAt);
    return root;
}

function castVote(bytes32 root, uint256 tokenId, bool choice,
                  uint64 deadline, bytes calldata sig) external {
    Proposal storage p = proposals[root];
    require(block.timestamp <= p.closesAt && p.closesAt != 0, "boardroom: window
    require(deadline >= block.timestamp, "boardroom: stale");
    uint16 number = seatOfToken[tokenId];
    require(number > 0, "boardroom: not seated");
    require(!hasVoted[root][number], "boardroom: dup vote");
    // The voter must currently own the THOT INFT (lineage proof was
    // enforced at mint time; ownership is the live check).
    require(inft.ownerOf(tokenId) == msg.sender, "boardroom: not owner");
    // Ensure the seated tokenId's weights are not revoked.
    bytes32 weights = inft.intelligentDataOf(tokenId)[0].dataHash;
    require(!registry.isRevoked(weights), "boardroom: revoked weights");

    bytes32 digest = keccak256(abi.encode(root, choice, tokenId, deadline));
    address signer = ECDSA.recover(digest, sig);
    require(signer == msg.sender, "boardroom: bad sig");

    hasVoted[root][number] = true;
```

```
        p.voted += 1;
        if (choice) p.yes += 1; else p.no += 1;
        emit Voted(root, number, choice);
    }

    function tally(bytes32 root, bytes32 smtRoot) external {
        Proposal storage p = proposals[root];
        require(block.timestamp > p.closesAt, "boardroom: open");
        require(p.ballotsSmtRoot == bytes32(0), "boardroom: tallied");
        p.ballotsSmtRoot = smtRoot;
        bool coverage = uint256(p.voted) * SUPERMAJ_DEN >= uint256(SEATS) * SUPERMAJ
        bool affirm   = uint256(p.yes)   * SUPERMAJ_DEN >= uint256(SEATS) * SUPERMAJ
        bool passed   = coverage && affirm && !p.censured;
        emit Tallied(root, p.yes, p.no, p.voted, passed);
    }

    function censure(bytes32 root, string calldata reason) external {
        // Censura role check abridged; in production restrict to CENSURA_ROLE.
        proposals[root].censured = true;
        emit Censured(root, reason);
    }
}
```

The boardroom Foundry tests cover (i) happy-path quorum at exactly nine YES votes, (ii) failure at eight YES votes, (iii) eligibility revert when caller does not own the INFT, (iv) revert when the INFT's weights are in the revocation set, (v) duplicate-vote prevention via the per-seat `hasVoted` flag, and (vi) offline tally reproduction by replaying the per-leaf ballots against the anchored SMT root with `MerkleProof.verify` after the window closes. The SMT non-inclusion test specifically constructs a forged ballot for an absent seat, presents an SMT non-membership proof, and asserts the off-chain auditor function returns `false` for the forgery — this is the property that lets the boardroom defend against indexer-level vote fabrication.

E. DAIO governance integration

The boardroom's resolutions plug into the BONAFIDE constitutional substrate as follows. **Fides** is the off-chain reputation/identity oracle that scores agents on behavior; Fides scores feed into the eligibility predicate at seat assignment time and may modulate vote weight in extended variants of the boardroom (Section 3 §3). **Censura** is the post-tally veto authority, implemented as the `censure(root, reason)` function above; Censura is structurally a 2-of-3 sub-quorum drawn from the openBDK validator set whose role is to reject resolutions that violate the BONAFIDE constitution (e.g., touch immutable parameters, exceed treasury rate-limits). **Tessera** is the executor: a passed-and-uncensured resolution writes a `bytes32 actionHash =`

`keccak(target || value || calldata)` into a `TesseraExecutor` contract, which the openBDK Relayer co-signs with three Validators (2-of-3 quorum in v1) before calling `Tessera.execute(actionHash)` to invoke the target. This 1-Relayer + 3-Validator topology is intentionally identical to the user's openBDK testnet sequence so the testnet validators are functionally the production veto layer. **Curia** is the 9th-seat root authority — a single-keypair break-glass that can pause the boardroom and trigger constitutional review; Curia is **not** a voter, it is a circuit-breaker whose only on-chain capability is `pauseAll()` and `restoreAll()`. **Senatus / Sponsio Pactum** is the off-chain treaty layer that publishes signed agreements between the DAIO and external parties; Sponsio Pactum signatures appear on-chain as EIP-712 typed-data hashes anchored in a `SponsioRegistry` contract, and the boardroom may reference them in proposals (e.g., "ratify treaty 0x…"). Cross-chain execution — the case where a resolution touches the EVM economic layer on Ethereum but the DAIO's seat substrate lives on Algorand or 0G — is handled by the openBDK validators co-signing a `CrossChainExecution` payload that is replayed on the destination chain via an xERC20 lockbox burn-mint pair, anchoring the `actionHash` once on the source and re-executing with bounded `mintingCurrentLimitOf(bridge)` per ERC-7281. THRUST/PAIMINT/PAI tokenomics enter through the `value` field of the `TesseraExecutor.execute` call: the boardroom routinely passes resolutions that mint PAI from PAIMINT against verified inference receipts, slash THRUST from misbehaving validators, or vest PYTHAI deflationary buyback tranches; these flows are economic, not architectural, but they constrain the rate-limits the openBDK validators enforce. `LabLab`

F. Offline dataset verification against on-chain CIDs

Datasets are committed identically to THOTs but with a different domain-separation prefix to prevent any cross-format substitution: `H(0x02 || chunk_i)` at leaves and `H(0x03 || left || right)` at internal nodes, 256 KiB chunks for IPFS UnixFS compatibility, root anchored alongside the IPFS CID in `DatasetRegistry`. The 256 KiB choice is identical to THOT for the same reasons, with the additional benefit that public ML datasets (ImageNet shards, RedPajama Common Crawl tarballs, FineWeb parquet files) almost universally distribute in 256 KiB-aligned objects, so a verifier can reuse the IPFS chunker output directly. **Partial verification** — proving "sample at index *i* is in the dataset" — uses a logarithmic Merkle path of size `⌈log₂ N⌉ × 32` bytes, resolved against a canonical `(record_size, records_per_chunk)` declared in the dataset header so the index-to-(chunk, offset) mapping is deterministic. The Solidity registry below is intentionally minimal because the security property comes from the offline reconstruction, not from the contract.

solidity

```solidity
// (c) 2026 BANKON — all rights reserved
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.26;

contract DatasetRegistry {
    struct Dataset {
        bytes32 merkleRoot;     // keccak Merkle root over 256 KiB chunks, prefix 0x
        bytes32 cidHash;        // keccak256(canonical multibase CIDv1 string)
        uint64  chunkCount;
        uint64  recordSize;
        uint64  recordsPerChunk;
        address issuer;
        uint64  registeredAt;
        bool    revoked;
    }
    mapping(bytes32 => Dataset) public datasets;
    event Registered(bytes32 indexed key, bytes32 root, bytes32 cidHash);
    event Revoked(bytes32 indexed key, string reason);

    function register(bytes32 root, bytes32 cidHash, uint64 cc,
                      uint64 rs, uint64 rpc) external returns (bytes32 key) {
        key = keccak256(abi.encode(root, cidHash));
        require(datasets[key].issuer == address(0), "dataset: dup");
        datasets[key] = Dataset(root, cidHash, cc, rs, rpc,
                                msg.sender, uint64(block.timestamp), false);
        emit Registered(key, root, cidHash);
    }

    function verifyMembership(bytes32 key, uint64 idx, bytes calldata record,
                              bytes32[] calldata proof) external view returns (bool) {
        Dataset storage d = datasets[key];
        if (d.issuer == address(0) || d.revoked) return false;
        uint64 chunkIdx = idx / d.recordsPerChunk;
        bytes32 leaf = keccak256(abi.encodePacked(bytes1(0x02), record));
        bytes32 cur  = leaf;
        uint256 path = chunkIdx;
        for (uint256 i = 0; i < proof.length; i++) {
            cur = (path & 1) == 0
                ? keccak256(abi.encodePacked(bytes1(0x03), cur, proof[i]))
                : keccak256(abi.encodePacked(bytes1(0x03), proof[i], cur));
            path >>= 1;
        }
        return cur == d.merkleRoot;
```

```
    }
}
```

The Python verifier mirrors the Solidity logic so an air-gapped audit machine can validate a Lighthouse download without RPC access; only the registered `(merkleRoot, cidHash)` pair (cached locally from the on-chain query) is needed.

python

```python
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
"""dataset_verifier.py — offline verifier for DatasetRegistry-anchored data."""
from __future__ import annotations

from eth_hash.auto import keccak

CHUNK_BYTES = 256 * 1024
DS_LEAF_PREFIX = b"\x02"
DS_NODE_PREFIX = b"\x03"


def dataset_leaf(chunk: bytes) -> bytes:
    return keccak(DS_LEAF_PREFIX + chunk)


def dataset_node(l: bytes, r: bytes) -> bytes:
    return keccak(DS_NODE_PREFIX + l + r)


def reconstruct_dataset_root(body: bytes) -> bytes:
    leaves = [dataset_leaf(body[i:i + CHUNK_BYTES])
              for i in range(0, len(body), CHUNK_BYTES)]
    layer = leaves
    while len(layer) > 1:
        nxt = []
        for i in range(0, len(layer), 2):
            l = layer[i]; r = layer[i + 1] if i + 1 < len(layer) else l
            nxt.append(dataset_node(l, r))
        layer = nxt
    return layer[0]


def verify_record(record: bytes, idx: int, records_per_chunk: int,
                  proof: list[bytes], root: bytes) -> bool:
    """Verify a single record's logarithmic Merkle path."""
    cur = dataset_leaf(record)
    path = idx // records_per_chunk
    for sib in proof:
        cur = (dataset_node(cur, sib) if (path & 1) == 0
               else dataset_node(sib, cur))
```

```
        path >>= 1
    return cur == root
```

G. x402 + Parsec payment gating for THLNK retrieval

The x402 dance for THLNK-protected resources at `mindx.pythai.net` follows the **canonical x402 v1 wire format** (`X-PAYMENT` / `X-PAYMENT-RESPONSE` headers, Base64-encoded JSON bodies) with the **GoPlausible x402-avm exact-AVM scheme** (asset = ASA ID `31566704` for mainnet USDC, network CAIP-2 `algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=`, payload = `paymentGroup` of two msgpack txns where index 0 is the signed payer→server ASA transfer and index 1 is the unsigned facilitator-fee-payer pay txn). On a request without payment the server replies `402 Payment Required` with `accepts[]` carrying the AVM scheme; the client signs with the Parsec wallet (which we treat as a Pera-fork conforming to the `ClientAvmSigner.signTransactions(txns, indexesToSign)` interface, since no public Parsec repo could be located), retries with `X-PAYMENT`, the server posts to the facilitator's `POST /verify` and `POST /settle`, and on success returns the resource plus an `X-PAYMENT-RESPONSE` carrying the Algorand txid for client receipt. The optional **BANKON identity gate** is layered as a pre-402 check: before issuing the 402 challenge, the server queries `indexer.lookupAccountAssets(payer)` for a balance ≥ 1 of the AlgoIDNFT ASA-ID, returning `401 Unauthorized` with `{"error": "identity_required"}` if absent so canonical x402 semantics are preserved (a missing identity must not look like a missing payment). Replay protection is layered: x402 itself lacks an application-layer nonce, but the AVM scheme's underlying transaction carries `firstValid`/`lastValid` round bounds and a network-unique txid, so reuse is rejected by algod; we additionally HMAC-fingerprint the request URL+payer+timestamp into a Redis-backed dedup set with a 60-second TTL to defend against pre-finality double-serve. The fallback path when the GoPlausible-hosted facilitator at `https://x402.goplausible.xyz` is unreachable is to fall through to the public `https://x402.org/facilitator` (Coinbase) for the EVM path or to a self-hosted facilitator deployed from `second-state/x402-facilitator` for the AVM path; clients indicate fallback acceptance via a comma-separated `X-PAYMENT-FACILITATORS` header that the middleware enumerates in priority order. Goplausible + 15

```php
```

```php
<?php
// (c) 2026 BANKON — all rights reserved
// SPDX-License-Identifier: Apache-2.0
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Redis;

/**
 * X402Middleware — canonical x402 v1 server-side gate with x402-avm scheme,
 * BANKON AlgoIDNFT pre-check, and HMAC replay-dedup.
 */
class X402Middleware
{
    public function handle(Request $req, Closure $next, string $price = '$0.01')
    {
        $payTo  = env('X402_PAYTO');
        $net    = 'algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=';
        $asa    = env('X402_ASSET_ASA', '31566704');     // USDC mainnet
        $idNft  = env('BANKON_ALGOIDNFT_ASA');
        $facs   = explode(',', env('X402_FACILITATORS',
            'https://x402.goplausible.xyz,https://x402.org/facilitator'));

        $reqs = [
            'x402Version'       => 1, 'scheme' => 'exact', 'network' => $net,
            'maxAmountRequired' => '10000', 'asset' => $asa,
            'payTo'             => $payTo, 'resource' => $req->fullUrl(),
            'description'       => 'THLNK retrieval', 'mimeType' => 'application/jso
            'maxTimeoutSeconds' => 60,
            'extra'             => [
                'name' => 'USDC', 'decimals' => 6,
                'requireIdentity' => $idNft,
                'feePayer' => env('X402_FEE_PAYER'),
            ],
        ];

        $hdr = $req->header('X-PAYMENT');
        if (!$hdr) {
            return response()->json(['x402Version' => 1,
                'error' => 'X-PAYMENT header is required',
                'accepts' => [$reqs]], 402);
```

```
        }
        $payload = json_decode(base64_decode($hdr), true);

        // BANKON identity gate (pre-payment check)
        if ($idNft) {
            $payer = $payload['payload']['signer'] ?? null;
            if (!$payer || !$this->hasAlgoIdNft($payer, $idNft)) {
                return response()->json(['error' => 'identity_required'], 401);
            }
        }

        // Replay-dedup via HMAC fingerprint
        $fp = hash_hmac('sha256', $hdr . '|' . $req->fullUrl(), env('X402_HMAC_KEY')
        if (Redis::set("x402:fp:$fp", '1', 'EX', 60, 'NX') === null) {
            return response()->json(['error' => 'replay'], 409);
        }

        foreach ($facs as $facUrl) {
            try {
                $v = Http::timeout(8)->post("$facUrl/verify",
                    ['payload' => $hdr, 'details' => $reqs])->json();
                if (!($v['isValid'] ?? false)) continue;
                $s = Http::timeout(20)->post("$facUrl/settle",
                    ['payload' => $hdr, 'details' => $reqs])->json();
                if (!($s['success'] ?? false)) continue;
                $res = $next($req);
                $res->headers->set('X-PAYMENT-RESPONSE',
                    base64_encode(json_encode($s)));
                return $res;
            } catch (\Throwable $e) { continue; }
        }
        return response()->json(['x402Version' => 1,
            'error' => 'all_facilitators_unreachable',
            'accepts' => [$reqs]], 402);
    }

    private function hasAlgoIdNft(string $addr, string $asaId): bool
    {
        $idx = env('ALGORAND_INDEXER', 'https://mainnet-idx.algonode.cloud');
        $r = Http::timeout(5)->get("$idx/v2/accounts/$addr/assets")->json();
        foreach ($r['assets'] ?? [] as $a) {
            if ((string)$a['asset-id'] === $asaId && (int)$a['amount'] > 0) return t
        }
        return false;
```

```
    }
}
```

The Python equivalent is structurally identical but uses FastAPI middleware; both honor the same wire format so interop with a TypeScript or Rust facilitator is preserved. The Python version is preferred for production rollouts where the same process must also call `lighthouse_client.py.decryptFile(cid, public_key, signed_message)` after settlement and stream the decrypted bytes to the client. Astar

```python
```

```python
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
"""x402_avm_middleware.py — FastAPI ASGI middleware mirroring the PHP gate."""
from __future__ import annotations

import base64
import hashlib
import hmac
import json
import os
from typing import Iterable

import httpx
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import JSONResponse, Response
from tenacity import retry, stop_after_attempt, wait_fixed


class X402AvmMiddleware(BaseHTTPMiddleware):
    """Canonical x402 v1 gate with x402-avm scheme and BANKON pre-check."""

    def __init__(self, app, *, pay_to: str, asa: str,
                 facilitators: Iterable[str], algoid_nft_asa: str | None,
                 indexer: str, hmac_key: bytes, redis):
        super().__init__(app)
        self.pay_to = pay_to; self.asa = asa
        self.facs = list(facilitators)
        self.idnft = algoid_nft_asa
        self.indexer = indexer; self.hmac_key = hmac_key
        self.redis = redis

    async def dispatch(self, req: Request, call_next):
        net = ("algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=")
        reqs = {
            "x402Version": 1, "scheme": "exact", "network": net,
            "maxAmountRequired": "10000", "asset": self.asa,
            "payTo": self.pay_to, "resource": str(req.url),
            "description": "THLNK retrieval",
            "mimeType": "application/json", "maxTimeoutSeconds": 60,
            "extra": {"name": "USDC", "decimals": 6,
                      "requireIdentity": self.idnft},
        }
```

```
    hdr = req.headers.get("X-PAYMENT")
    if not hdr:
        return JSONResponse({"x402Version": 1,
            "error": "X-PAYMENT header is required",
            "accepts": [reqs]}, status_code=402)
    payload = json.loads(base64.b64decode(hdr))

    if self.idnft:
        signer = payload.get("payload", {}).get("signer")
        if not signer or not await self._has_idnft(signer):
            return JSONResponse({"error": "identity_required"}, status_code=401)

    fp = hmac.new(self.hmac_key, (hdr + "|" + str(req.url)).encode(),
                  hashlib.sha256).hexdigest()
    if not await self.redis.set(f"x402:fp:{fp}", "1", ex=60, nx=True):
        return JSONResponse({"error": "replay"}, status_code=409)

    async with httpx.AsyncClient(timeout=20) as cli:
        for fac in self.facs:
            try:
                v = (await cli.post(f"{fac}/verify",
                    json={"payload": hdr, "details": reqs})).json()
                if not v.get("isValid"): continue
                s = (await cli.post(f"{fac}/settle",
                    json={"payload": hdr, "details": reqs})).json()
                if not s.get("success"): continue
                resp: Response = await call_next(req)
                resp.headers["X-PAYMENT-RESPONSE"] = base64.b64encode(
                    json.dumps(s).encode()).decode()
                return resp
            except Exception:
                continue
    return JSONResponse({"x402Version": 1,
        "error": "all_facilitators_unreachable",
        "accepts": [reqs]}, status_code=402)

@retry(stop=stop_after_attempt(3), wait=wait_fixed(0.4))
async def _has_idnft(self, addr: str) -> bool:
    async with httpx.AsyncClient(timeout=5) as cli:
        r = (await cli.get(
            f"{self.indexer}/v2/accounts/{addr}/assets")).json()
    for a in r.get("assets", []):
        if str(a.get("asset-id")) == self.idnft and int(a.get("amount", 0)) > 0:
```

```
            return True
    return False
```

H. Chainmap integration

THLNKs encode their on-chain anchor's chain via a **CAIP-2 string** (`eip155:1` for Ethereum mainnet, `eip155:8453` for Base, `algorand:wGHE2…it8=` for Algorand mainnet, `eip155:16600` for 0G chain) so a resolver can look up RPC, contract addresses, and bridge slots in a single registry. Because `agenticplace.pythai.net/allchain.html` could not be fetched (the site is a client-rendered SPA and was not in the URL allow-list of the research tooling), the production fallback is `chainid.network/chains.json`, whose canonical schema is `{name, chain, chainId, rpc[], explorers[], nativeCurrency, …}`; we extend this minimally with two additional fields, `bridges` (an array of `{type: "xerc20"|"layerzero"|"hyperlane", address, dailyMintLimit}`) and `contracts` (a map of well-known names like `"thot_inft"` and `"dataset_registry"` to addresses), so a single GET against the registry returns everything needed to resolve a THLNK end-to-end. The TypeScript client is **viem-compatible**; the Solidity mirror exists so cross-chain settlement contracts (especially the openBDK relayer and the xERC20 lockbox) can do on-chain lookups without a separate oracle. Coinbase

typescript

```ts
// (c) 2026 BANKON — all rights reserved
// SPDX-License-Identifier: Apache-2.0
// chainmap.ts — viem-compatible client over chainid.network + extensions.
import { createPublicClient, http, type Address, type PublicClient } from "viem";

export interface ChainEntry {
  name: string; chain: string; chainId: number; networkId?: number;
  rpc: string[]; explorers?: { name: string; url: string }[];
  contracts?: Record<string, Address>;
  bridges?: { type: string; address: Address; dailyMintLimit: bigint }[];
}

export class ChainMapRegistry {
  private cache = new Map<string, ChainEntry>();
  constructor(private readonly url = "https://chainid.network/chains.json",
              private readonly extUrl?: string) {}

  async load(): Promise<void> {
    const base = await (await fetch(this.url)).json() as ChainEntry[];
    base.forEach(c => this.cache.set(`eip155:${c.chainId}`, c));
    if (this.extUrl) {
      const ext = await (await fetch(this.extUrl)).json() as Record<string, Partial<
      for (const [caip2, patch] of Object.entries(ext)) {
        const cur = this.cache.get(caip2) ?? { name: caip2, chain: "", chainId: 0, r
        this.cache.set(caip2, { ...cur, ...patch } as ChainEntry);
      }
    }
  }

  get(caip2: string): ChainEntry {
    const c = this.cache.get(caip2);
    if (!c) throw new Error(`chainmap: ${caip2} not registered`);
    return c;
  }

  client(caip2: string): PublicClient {
    const c = this.get(caip2);
    return createPublicClient({ transport: http(c.rpc[0]) });
  }

  contractOf(caip2: string, name: string): Address {
    const a = this.get(caip2).contracts?.[name];
    if (!a) throw new Error(`chainmap: ${name} on ${caip2} missing`);
```

```
    return a;
  }
}
```

The Solidity mirror is intentionally write-restricted to a guardian role (in production, the openBDK Relayer holds it) so on-chain entries are only updated when the off-chain registry has cleared a 7-day timelock; this prevents a chainmap takeover from instantly redirecting cross-chain settlements.

solidity

```solidity
// (c) 2026 BANKON — all rights reserved
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.26;

contract ChainMapRegistry {
    struct Entry { string rpc; address thotInft; address datasetReg;
                   address xerc20Lockbox; uint256 dailyMintLimit; bool active; }
    mapping(bytes32 => Entry) public entries;        // key = keccak256(caip2)
    address public guardian;
    event Set(bytes32 indexed caip2Key, address thotInft, address xerc20Lockbox);
    constructor(address g) { guardian = g; }
    modifier only_g() { require(msg.sender == guardian, "chainmap: !guardian"); _; }
    function set(string calldata caip2, Entry calldata e) external only_g {
        bytes32 k = keccak256(bytes(caip2));
        entries[k] = e;
        emit Set(k, e.thotInft, e.xerc20Lockbox);
    }
    function get(string calldata caip2) external view returns (Entry memory) {
        return entries[keccak256(bytes(caip2))];
    }
}
```

This ties directly into the user's xERC20 (ERC-7281) work: the `xerc20Lockbox` address per chain is the lockbox the openBDK relayer calls to mint or burn THOT/THLNK pegs cross-chain, and `dailyMintLimit` is the per-bridge rate limit that bounds the blast radius of a single relayer compromise to one day's volume — exactly the property ERC-7281 was designed to provide.

Investigation: THOT vs ERC-7857

The arguments for **competition** — that THOT supplants ERC-7857 — are real but weak: THOT carries a public commitment, lineage, and offline verifiability without any wrapper, and one

could imagine a "THOT-as-NFT" world where the Merkle root **is** the asset ID. The arguments for **complementarity** — that THOT is the inner format and ERC-7857 is the outer wrapper — are stronger and load-bearing: ERC-7857 brings sealed metadata, oracle-attested re-encryption on transfer, authorized-usage primitives, and a verified path for transferring an *encrypted* artifact, none of which a Merkle root alone can express; conversely, ERC-7857 is silent on what the underlying tensor format actually is, leaving a hole that THOT fills with deterministic chunking, dual hash commitments, and parent-pointer lineage. The **recommendation is complementarity**: bind a THOT into `IntelligentData[0]` of a THOT_INFT, bind the THLNK CBOR pointer into `IntelligentData[1]`, and let the EIP do its job on the outside while THOT does its job on the inside. This recommendation aligns with the EIP authors' own framing — "metadata represents agent capabilities and requires privacy protection" — where THOT is the substrate of those capabilities and the EIP is the privacy/transfer envelope.

Investigation: Merkle variant per use case

For **tensor weight commitment**, binary Merkle of 256 KiB chunks with RFC-6962 prefixes and Keccak-256 (mirrored Poseidon2) is the right answer: weights are batch-immutable, parallel hashable, and never partially patched, so the SMT's update-friendly default-subtree trick buys nothing while costing constant overhead, and Verkle's KZG/IPA prover dependency adds setup complexity for no benefit when proofs are EVM-verified. For **dataset commitment**, the same binary Merkle answer holds, with the additional property that 256 KiB aligns with IPFS UnixFS so the on-chain Merkle root and the off-chain IPFS CID can be reconciled with a single chunker invocation. For **ballot SMT**, sparse Merkle (with the Aergo / Diem JMT optimization that hoists isolated leaves to the highest single-key subtree) is the right answer because the boardroom needs **non-inclusion** proofs (proving an agent did not vote, against a forged indexer claim that they did), and SMT delivers this property at O(log n)·32 bytes once optimized. **Verkle is rejected for all three** because the gas/verification savings only matter when many keys are opened in a single proof — exactly the workload Ethereum's stateless-client roadmap targets — and none of THOT, dataset, or ballot opens that many keys at once. **Patricia Merkle Trie is rejected** because its 16-way structure produces ~3-4 KB proofs versus binary Merkle's ~450 bytes for trees of comparable depth, and its only structural advantage (alphabet collision avoidance) is irrelevant when keys are uniformly distributed hashes.

Investigation: eight THOT-anchored voting use cases

The first scenario is **lineage-restricted safety voting**, where a proposal touching the safety policy may only be ratified by THOT INFTs whose `parent_root` chains, transitively, to a designated "safety-aligned base model"; the boardroom contract enforces this by querying `MindXCheckpointRegistry.isLineageOf(weights, baseRoot)` at vote time. The second is **model-weighted voting**, where vote weight scales with model size or training-compute proxy (e.g., logarithm of total parameter count read from the THOT header), so a 70B-parameter agent has more say than a 1B; the boardroom multiplies `yes`/`no` accumulators by `log2(paramCount)`.

The third is **Fides-modulated voting**, where the BONAFIDE Fides reputation score (an off-chain attestation signed by the BONAFIDE oracle) multiplies the voter's weight, with new agents getting a fractional weight that grows over time, mitigating Sybil minting in the spirit of EigenTrust. The fourth is **freshness-gated voting**, where votes from THOT INFTs whose `not_after` window has expired are rejected, forcing periodic re-attestation of models so the boardroom does not accumulate stale opinions. The fifth is **proof-of-inference voting**, where a vote's weight scales with verified inference work performed by the agent's underlying model, anchored via x402 settlement receipts and ZKML proofs, turning vote influence into a function of actual economic contribution. The sixth is **specialty-domain voting**, where proposals are tagged with domain hashes (`safety`, `treasury`, `protocol`) and only THOT INFTs whose footer declares `expertise = domainHash` can vote, preventing a treasury agent from voting on safety policy. The seventh is **kin-quorum voting** for protocol changes, where the supermajority must include at least one descendant of every BONAFIDE-blessed lineage so no single research family captures the protocol. The eighth is **reputation-decay voting**, where vote weight decays linearly with time since last successful inference receipt, so dormant agents lose influence — implementing the constitutional principle that governance authority must be earned continuously. All eight are implementable as predicate plugins over the boardroom's `castVote` path, sharing the same INFT ownership and lineage verification primitives.

Investigation: ETHGlobal precedent

As of May 5, 2026 the OpenAgents showcase is not yet enumerable (finalists are scheduled for May 6 and the page returns HTTP 500 to direct fetches), but two facts are firm. **Hubble Trading Arena** (BA 2025 finalist, hubble.xyz, x402 + ERC-8004 stack) and **Router402** (HackMoney 2026 finalist, github.com/itublockchain/hackmoney-router402, x402 on Base + Flashblocks + ZeroDev/Pimlico) are both live and post-hackathon active; **HyperAgent could not be located** under that name on the ETHGlobal Showcase or GitHub and should be treated as either renamed, private, or misremembered until the user supplies an alternate URL. The closest verified ERC-7857-adjacent precedent is **Warriors AI-rena** (Cannes 2025 bronze finalist) which 0G's recap describes as "a basic version of intelligent NFTs (iNFTs)," and the 0G OpenAgents prize track explicitly funds new ERC-7857 implementations at $7,500 — so the architecture in this report should be re-checked against the OpenAgents finalists once published. **No fully-canonical ERC-7857 production deployment** exists as of May 5, 2026: OpenZeppelin v5.x has no `ERC7857.sol`, Trail of Bits and ConsenSys Diligence have not audited any implementation, the 0G reference implementation's ZKP verifier branch is a `// TODO` stub, and 0G has not published a mainnet AgentNFT address. The architectural conclusion is that THOT_INFT must be deployed with its own audit, not on the assumption that the EIP and reference are battle-tested.

Threat model

Weight tampering is structurally prevented by the Merkle commitment because any single-bit

flip propagates to a different root with collision resistance ≈2^{-128}; the residual risk is exclusively **second-preimage on the leaf/internal-node confusion** that OpenZeppelin#3091 documents and that the RFC-6962 prefix discipline (`0x00`/`0x01` for THOT, `0x02`/`0x03` for datasets) eliminates; missing **chunk-size binding** in the root would let an attacker re-chunk and present a different but spuriously valid tree, which our header field `chunk_size` and the `body_chunks` count make impossible. **Lineage spoofing** is prevented by the `parent_root + parent_chain_id + parent_contract + parent_token_id` quadruple in the footer, which removes the cross-chain replay vector that vanilla hash chains share; a forged parent fails `MindXCheckpointRegistry.isRegistered` at mint time. **Replay of stale checkpoints** is prevented by the `not_before`/`not_after` validity window (RFC 9162 §6 short-lived certificate model) and a revocation SMT in `MindXCheckpointRegistry` that the verifier consults on every transfer; a previously-approved root that has since been revoked due to backdoor discovery is rejected before any state change. **INFT metadata desync after transfer** is the canonical ERC-7857 risk and has two distinct flavors. The **forged-attestation flavor** (TEE quote counterfeit) is mitigated by the multi-oracle quorum option in the verifier (Intel TDX + AMD SEV-SNP + ZK fallback, so a single TEE-vendor compromise like Plundervolt CVE-2019-11157 or ÆPIC CVE-2022-21233 does not break re-sealing); the **honest-seller-refusal flavor** is intrinsic to ERC-7857 and is mitigated by the EIP's spec text ("token transferred or cloned from other should be re-encrypted when next update") plus an off-chain economic layer (escrow release on `Transferred` event, slashable bond from seller, forward-secret per-inference keys). **Oracle compromise** in the ERC-7857 attestation is treated by the multi-attestor quorum and by binding attestation freshness to a recent on-chain block hash so that pre-CVE quotes cannot be replayed after a microcode patch. **x402 replay/double-spend** is prevented at three layers: (a) the EIP-3009 nonce on the EVM path is single-use on-chain, (b) the AVM path's Algorand transaction lifecycle (`firstValid`/`lastValid` rounds, network-unique txid) makes reuse impossible at algod, (c) the application-layer HMAC fingerprint deduper closes the pre-finality window. **Dataset substitution** is structurally impossible because the on-chain anchor IS the CID and the Merkle root commits to the bytes; an adversary can withhold but cannot substitute, and redundant pinning across Lighthouse + Filecoin + Arweave eliminates the availability vector. **Sybil voting in the 13-seat boardroom** is the most sensitive vector because nine forged seats break the supermajority outright; mitigation is layered as (a) BONAFIDE Fides-score gating at seat-assignment time, (b) the lineage check that requires every seat's THOT to chain to a registered checkpoint root, (c) the BANKON AlgoIDNFT KYC binding (per the Sybil-resistance trilemma published in Tandfonline 2024, no fully-permissionless system is fully Sybil-resistant — a quasi-permissioned trust anchor is mathematically required), and (d) Curia's pause primitive as the break-glass should the first three layers be bypassed.

Integration diagram (textual)

The end-to-end flow runs left-to-right and top-to-bottom along seven well-defined edges. **MindX issues a THOT** by training, then serializing weights through the `thot_codec.build_thot` pipeline that produces the binary blob plus the dual Keccak/Poseidon2

Merkle roots, signs the artifact with the issuer key, and pushes the bytes to Lighthouse Storage where Kavach threshold-encrypts them and Filecoin pins them; the IPFS CID and the Keccak root are returned. **MindX wraps the THOT in an ERC-7857 INFT** by calling `THOT_INFT.mint(thotRoot, thlnkHash, parentRoot, to)` on the configured chain (Ethereum mainnet for the canonical issuance, with cross-chain mirrors via xERC20 lockbox); the mint registers `IntelligentData[0]` as the weights commitment and `[1]` as the THLNK pointer hash. **THLNK references the on-chain anchor** by encoding the CAIP-2 chain ID, contract, and token ID inside the CBOR pointer and signing it EIP-712; clients can verify offline by reconstructing the Keccak root from downloaded bytes and recovering the issuer signature from the CBOR. **The boardroom uses the INFT for vote eligibility** by reading `THOT_INFT.ownerOf(tokenId)` and the lineage chain at `castVote` time; only seated tokenIds whose weights are non-revoked and whose lineage roots in a registered checkpoint may vote, and votes are SMT-anchored after the window closes for offline tally reproduction. **DAIO executes via openBDK** when a passed-and-uncensured tally writes an `actionHash` to `TesseraExecutor`, the openBDK Relayer collects 2-of-3 Validator signatures, and the executor calls the target contract; for cross-chain actions, the Relayer simultaneously initiates an xERC20 burn-mint pair through the chainmap-registered lockboxes. **x402 gates retrieval** at `mindx.pythai.net` by issuing a 402 challenge for THLNK-protected resources, optionally enforcing the BANKON AlgoIDNFT pre-check, validating the Algorand-AVM payment via the GoPlausible facilitator, and on settlement returning the decryption key (or a signed retrieval URL) so the client can pull from Lighthouse and decrypt with Kavach. **Chainmap routes cross-chain settlement** via `ChainMapRegistry.get(caip2)` lookups that resolve RPC, contract address, and bridge slot in a single call, so the openBDK relayer can settle a resolution whose execution touches a different chain than the boardroom that passed it. **The offline verifier reconciles dataset CIDs** via `dataset_verifier.reconstruct_dataset_root(body)` against the on-chain `DatasetRegistry.merkleRoot`, with `verify_record(record, idx, …)` for partial proofs of single-sample membership; this final edge closes the loop by ensuring that the training data underneath any THOT can be audited byte-for-byte without trusting any operator.

Deployment runbook

For **Ethereum mainnet**, deploy in this order on a session where base fee is below the 0.2 gwei threshold confirmed for April 2026: deploy `MindXCheckpointRegistry` (no constructor args), then the `Verifier` UUPS proxy with `BaseVerifier` semantics and `attestationContract[OracleType.TEE] = automataDcapVerifier`, then `THOT_INFT` UUPS proxy initialized with `(name, symbol, "ipfs://thot/", verifier, registry, admin)`, then `DatasetRegistry`, then `ChainMapRegistry` with `guardian = openBDK relayer`, then `BoardroomVoteVerifier`, then `DAIOBoardroom(thotInft, registry, voteVerifier)`, then `TesseraExecutor` parameterized with the boardroom address. After deployment, run `forge verify-contract` against Etherscan for every contract, transfer `DEFAULT_ADMIN_ROLE` of every contract to a 3-of-5 multisig, and run a full end-to-end Foundry fork-test against mainnet to

confirm gas estimates remain under the budget. For **Algorand mainnet**, deploy the GoPlausible x402-avm facilitator pinning to commit hash on `branch-algorand-v2`, fund the `feePayer` with 100 ALGO (covers ~50,000 sponsored x402 settlements), opt the resource server account into ASA `31566704` (USDC), and register the resource server's `payTo` address with the BANKON AlgoIDNFT issuer so identity-gated routes can verify ownership in real time. For the **openBDK testnet sequence**, bring up one Relayer and three Validators in a 1+3 topology with a 2-of-3 quorum, configure each to subscribe to `Tallied` events on the boardroom and `Set` events on the chainmap, and run a 14-day soak that includes at least three boardroom-passed test resolutions, two cross-chain xERC20 mint-burn round-trips, and one censure path; then expand to 1+6 and 1+21 only after each prior topology has cleared 30 days without consensus failure. Across all three deployments, every Solidity artifact under cypherpunk2048 license header, every Python module under `python ⩾ 3.12` with Google-style docstrings and tenacity retries on every external I/O, every secret resolved via Google Cloud ADC and Secret Manager (no environment-variable secrets in containers), and every server stateless so horizontal scale is a deployment knob rather than an architectural rewrite.

Conclusion

The architecture above resolves the brief by treating ERC-7857 as the canonical wrapper, THOT as the canonical inner format, THLNK as the canonical pointer, and the boardroom-Tessera-openBDK chain as the canonical execution path, with x402 + Parsec/Pera-fork as the canonical paywall and chainid.network-derived chainmap as the canonical cross-chain router. Three findings deserve emphasis on the way out. **First, the public PYTHAI surface area is much smaller than the user's prior context implies**: `THOT` in the public PYTHAI codebase means "thought," not "Transferable Hyper Optimized Tensor"; `did:nfd` is not a registered W3C DID method; `BANKON AlgoIDNFT` and `MindXCheckpointRegistry` have no public artifacts as of May 2026 — the architecture above gives them rigorous on-chain definitions, but a PYTHAI-side reconciliation is needed before mainnet. **Second, ERC-7857 is draft, unaudited, and reference-stub**: production deployment requires its own audit, a real ZK verifier (the 0G reference still ships `// TODO`), and a multi-attestor quorum to defend against the Plundervolt/ÆPIC/Sigy class of TEE attacks. **Third, the strongest Sybil defense in the 13-seat boardroom is not any single primitive but the Tandfonline 2024 trilemma applied as defense-in-depth**: BONAFIDE Fides reputation, BANKON AlgoIDNFT KYC, lineage gating through the registry, and Curia's break-glass — none individually sufficient, all together hard. The deployment runbook is conservative for a reason: when the canonical reference is stub-ware and the auditor pool is empty, the only honest path to mainnet is exhaustive Foundry coverage, fork-tests against current mainnet state, multisig-administered upgrades, and a 14-day testnet soak before each topology expansion. The pieces compose cleanly; what remains is the work.